Numerical Methods for Non-Linear Mechanics

Viet Anh Quach

November 13, 2025

1 Introduction

This short report present the code that is assign for the subject "Numerical Methods for Non-Linear Mechanics" of the Master Program for Civil Engineering, teached by professor Stafano-Dal Pont at Laboratory 3SR, University Grenoble Alpes, France. The object of the course is studying the non-linear problems in mechanics, caused by both the geometric and material behavior. The assigment practices therefore is to build in the most general form of the constitutive law, which is not linearity. 2 main problems are takin into account: A console problem in 2D and a Truss problem in 3D, so the remaining of this reports are divided into two sections of the former subject. Besides, this is a ideal chance to practice, so I also try to provide a solution for the problem of Finite Element Method in 1D and 2D. The main code is actually written in `C++`, with libraries mostly building from scratches. They have been upload to `Github`, the link to download is provided in **??**. Any corrections and comments are highly appreciated. I would like to thanks the professors for letting me, even though being a PhD student, could join this program to learn. I really appreciated and enjoyed the class.

2 Theory

The following problem was fully described in lecture course [**?**].

2.1 Console

The whole geometry is described in fig. 1. Breifly speaking, the simple truss structure composed by 3 nodes linked by 2 bars and external force F applied on the node C maing an angel θ with the vertical axis, causing the displacement of the bars and nodes. Gravity is not taken into account. In this case, $\vec{x}$ representing position of node C is considered as the main unknown, owing to the fact that it is the only free node and fully represent the equilibrium position of the system. $\hat{e}^b$ is the unit vector of the corresponding bar b, which is calculated by:

$$\hat{e}^b = \frac{\vec{b}}{l^b} \tag{1}$$

In truss elements, the only internal effort considered is the axial stress and further, also is a constant. N^b denotes this componeet that the right part exerts on the left part of bar b, which is assumed to depend only on the length of bar l^b via the constitutive law:

$$N^b = \mathcal{A}^b(l^b) \tag{2}$$

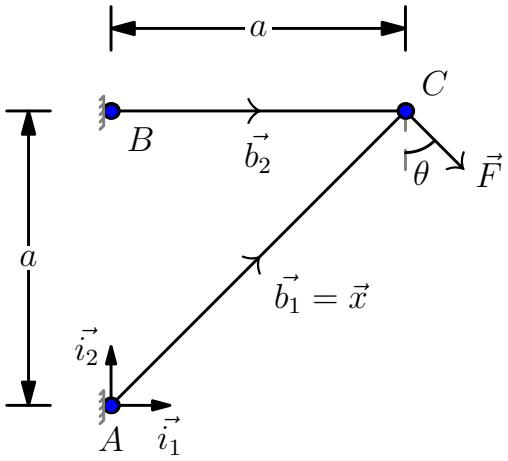

Figure 1: Two elements truss: console 2D

In case of large displacelents, this relation is no longer linear, as the length of the bar changed accordingly with the new position of the node, in our case is node C. So the problem of the console, can be written as follows:

$$\text{Given the force } \vec{\mathcal{F}}, \tag{3}$$

$$\text{Find the position } \vec{x} \text{ of node C such that,} \tag{4}$$

$$-\mathcal{A}^1(l^1(\vec{x}))\hat{e}^1(\vec{x}) - \mathcal{A}^2(l^2(\vec{x}))\hat{e}^2(\vec{x}) + \vec{\mathcal{F}} = \vec{0} \tag{5}$$

This problem is solved by Newton's iterative method in vector form for solving systems of equations. For the order-one truncated Taylor series expansion, expression at the iteration (k+1) shall be:

$$\underline{\underline{\nabla \mathcal{F}}}(\underline{x}^{(k)})\delta(\underline{x}^{(k)}) = -\underline{\mathcal{F}}(\underline{x}^{(k)}) \tag{6}$$

Solving the linear system get us the new value of δx, so we could update the postion for the next iteration by:

$$\underline{x}^{(k+1)} = \underline{x}^{(k)} + \delta\underline{x}^{(k)} \tag{7}$$

Apply to the console problem eq. (6) becomes The method requires a stopping criterion: eq. (8) is chosen to be implemented in the code, as it satisfies the force equilibrium condition at nodes of the problem.

$$\left|f\left(x^{(k)}\right)\right| < \epsilon \tag{8}$$

As ϵ is a number small enough to be considered as 0, relatively to the problem solving. The object is now to transform equation eq. (2) into a linear form eq. (6) so that it could be numerically solved. Thanks to first-order Taylor expansion again, we obtain the development of sum of the force, which should be approching 0, at iteraion `k+1` (or in another word, at when node **C** would take an increment in position into $\vec{x} + \delta\vec{x}$):

$$\vec{\mathcal{F}}(\vec{x} + \delta\vec{x}) = \vec{\mathcal{F}}(\vec{x}) + \left(\nabla\vec{\mathcal{F}}(\vec{x})\right) @\delta\vec{x} = 0 \tag{9}$$

where the Jacobian matrix $\nabla\vec{\mathcal{F}}(\vec{x})$ is described in the following euqation using Einsten's sum notation:

$$\nabla\vec{\mathcal{F}}(\vec{x}) = -\frac{d\mathcal{A}^i}{dl^i}\left(\vec{e}^i(\vec{x}) \otimes \vec{e}^i(\vec{x})\right) - \frac{\mathcal{A}^1\left(l^i(\vec{x})\right)}{l^i(\vec{x})}\left(\mathbb{I} - \vec{e}^i(\vec{x}) \otimes \vec{e}^i(\vec{x})\right) \tag{10}$$

Finally, we can solve the system using a single linear vector equation, which is suitable to be implemented into computational calculating:

$$\left(\nabla \vec{\mathcal{F}}(\vec{x}^k)\right) @ \delta \vec{x}^k = -\vec{\mathcal{F}}(\vec{x}^k) \tag{11}$$

The first iteration starts from the initial condition of the system, then the iteration begins until $\vec{\mathcal{F}}(\vec{x}^k) < \epsilon$ to be considered null, as discussed above.

2.2 Truss

3 Validation

3.1 Console

3.2 Truss

4 Application

4.1 Console

4.2 Truss

5 Conclusion

6 Annexxe

This part just provided the main code. Full version could be download in this link.s